

ChronoPace

Real-Time Energy & Overtake Strategy Intelligence for 2026 Formula 1

Solution Design Document · TrackShift 2026 Innovation Challenge (AI Motorsport Intelligence) · 29 August 2026

This document explains the problem ChronoPace solves, the full solution — the decision engine, not just the dashboard on top of it — and the reasoning behind every major design choice, so the whole team is working from the same understanding before we divide up the remaining build.

1. Problem Statement

Under the old hybrid-era rules, the electrical system supplied roughly 20% of a Formula 1 car's power output, and energy deployment was a secondary tactic. 2026 changes that fundamentally: the MGU-H is removed, electrical power rises to roughly a 50/50 split with the internal combustion engine, and DRS is replaced by a proximity-gated, delayed energy allowance ("Overtake Mode"). Deployment timing is no longer a minor optimization — it is the primary strategic lever, and there was no equivalent decision to make under the previous regulations.

Concretely, a race strategist now has to answer, lap after lap, under time pressure:

- **Is this move even legal right now?** Energy caps, delta-SoC limits and Overtake Mode eligibility all depend on live telemetry, and getting this wrong is a regulatory breach, not just a bad call.
- **Is it actually worth doing?** A legal option can still be a bad bet — outcomes are uncertain, and "legal" and "good idea" are different questions answered by different evidence.
- **How confident are we, really?** A marginal, noisy edge is not the same as a real one, and acting on noise loses more races than it wins.
- **What does the car we're racing actually have?** No team publishes ERS state of charge — not even to itself for a rival — so a rival's true energy reserve is fundamentally hidden.

This is squarely the official brief's first theme — AI Motorsport Intelligence — and it is a decision problem, not a data-visualization problem: the hard part is producing a strategy call that is simultaneously legal, statistically justified, and honest about its own uncertainty, fast enough to act on.

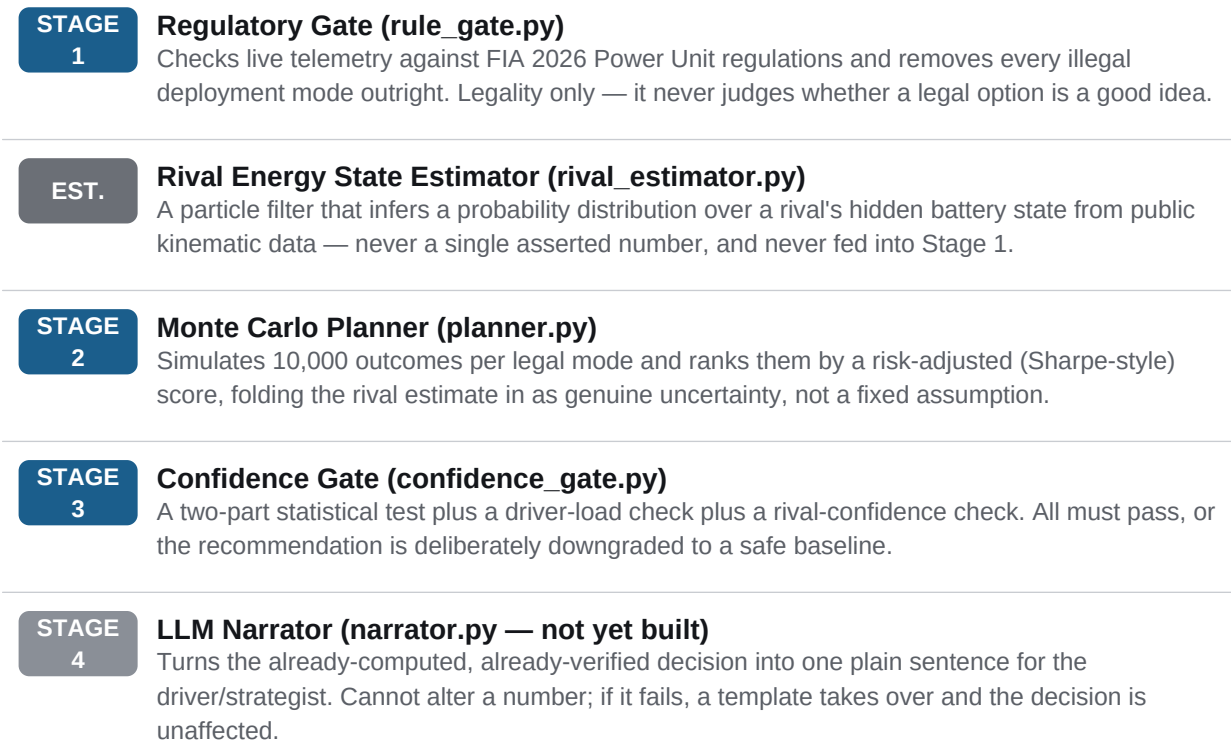
2. The Solution — ChronoPace

ChronoPace is a four-stage deterministic decision pipeline, plus one supporting estimator module. Every stage is a plain Python computation with a fixed, typed input/output contract (Pydantic models) — nothing probabilistic about what a stage *does*, only about the racing outcomes it reasons over.

The one architectural law

Python computes every number, deterministically and testably. A language model is only ever allowed to turn an already-correct, already-verified result into a plain-language sentence. It never computes, never decides, and never has authority to change a number it was given. This is not a style preference — a language model generates tokens probabilistically and can state a wrong number with full confidence, which is not acceptable when the number is an energy deployment figure or a compliance decision. No language model call exists anywhere upstream of the final decision.

Pipeline overview



Stage 1 output feeds Stage 2; Stage 2's ranked output plus the rival estimate feed Stage 3; Stage 3's gated decision is what Stage 4 will eventually narrate. Each arrow is a fixed, typed contract — no stage recomputes or second-guesses the one before it.

The five deployment modes

Under 2026 rules, the +0.5 MJ Overtake Mode bonus is banked on the lap a car qualifies (within 1.0s of the car ahead at the detection point) and can only be spent on the *following* lap. That is a genuine sequential-decision problem — the decision to pursue the bonus is made a full lap before the benefit exists — so "arming" and "spending" are modeled as two distinct modes, not one mode with a hidden flag.

Mode	What it represents	Legality condition
CONSERVE_MODE	Minimal deployment, banks energy for later	Base cap only
BALANCED_MODE	Default baseline; Stage 3's safe fallback target	Base cap only

Mode	What it represents	Legality condition
ARM_OVERTAKE_MODE	Attack this lap while within range of the car ahead; qualifies next lap's bonus	Base cap, and within the 1.0s gap threshold
USE_OVERTAKE_BONUS_MODE	Spend a bonus banked from the previous lap	Base cap + bonus, and bonus was banked last lap
PUSH_MODE	Sustained max-legal deployment (e.g. defending)	Base cap only

Regulatory grounding

Every energy-related constant is anchored to a cited FIA 2026 Power Unit Technical Regulation article, not an invented number — specifically so a reviewer can check each figure against the actual regulation text without reading implementation logic.

Constant	Value	Source
Max MGU-K power	350 kW	Art. 5.4.7
Max deployment per lap	9.0 MJ	Art. 5.4.10
Max delta-SoC swing per lap	4.0 MJ	Art. 5.4.9
Overtake Mode eligibility gap	1.0 s at detection point	F1 Sporting Regulations
Overtake Mode bonus	+0.5 MJ, following lap only	FIA / F1 official

One exception, flagged deliberately rather than silently rounded: the per-lap recoverable-energy baseline is event-variable (as low as 5 MJ at energy-starved circuits, up to 9 MJ at energy-rich ones, 7 MJ in qualifying) and its precise article citation is still pending confirmation against the primary regulation text — we would rather say that plainly than invent a citation.

The Rival Energy State Estimator

No team publishes ERS state of charge, including to rivals. An earlier idea — inferring a rival's battery state from its engine's acoustic signature — was evaluated and rejected: audio can at best hint at a rival's *instantaneous deployment rate*, never its *stored, integrated energy*, and separating one car's signature from nineteen others through broadcast audio compression is an unsolved source-separation problem, not an engineering narrowing problem.

The estimator instead runs a particle filter over two public, kinematic observables every team already receives from FIA timing/GPS — a rival's terminal straight-line speed, and where on the straight its speed trace flattens ("clipping point"). Its output is always a posterior **distribution** (a mean and a standard deviation), never a single point value asserted as fact, and it is architecturally barred from ever reaching Stage 1: the module may only import shared constants from the regulatory gate, never the other way around, so a rival's estimated state can never influence what is or isn't legal.

3. Why We Chose This Approach

Each of the decisions below was deliberated against a real alternative — they are recorded here so the reasoning travels with the team, not just the result.

Deterministic Python over an LLM or RL agent deciding

An LLM can state a wrong number with full confidence, which is unacceptable for an energy or compliance decision. Reinforcement learning needs an environment to learn in, and our environment would be our own simulator — an RL agent would learn our modeling assumptions, not the sport, and would be unexplainable at exactly the moment a strategist needs to know why. Monte Carlo rollouts over an explicitly bounded, legality-checked action space give the same search with a full audit trail.

A hard-prune legality gate, not a penalty function

An illegal action is not "expensive" in the model — it does not exist as an option at all. This is a genuine engineering distinction most naive designs get wrong (a penalty function still lets an optimizer choose an illegal action if the expected reward is high enough); a hard prune cannot.

Monte Carlo with risk-adjusted (Sharpe-style) ranking, not expected value alone

Ranking purely by average outcome hides how risky each option is. A risk-adjusted score captures the outcome-vs-variance trade-off a real strategist actually cares about, and it degrades gracefully — near-zero variance is explicitly capped rather than returned as an undefined or infinite value, so every downstream statistic stays well-behaved.

A two-part confidence gate, not a plain significance test

A plain statistical significance test is gameable by simply running more simulation rollouts: standard error shrinks as the simulation count grows, so at a large, fixed rollout count almost any nonzero difference between two options clears a naive significance bar, whether or not the difference is big enough to actually matter. The fix — the standard superiority-against-a-margin construction used in clinical trials — requires a result to be both statistically reliable *and* larger than a stated minimum actionable effect before it is trusted, with the rollout count fixed and reported so it cannot be tuned to manufacture significance.

Abstention as a first-class output, not a failure mode

When the evidence is thin, ChronoPace deliberately falls back to a safe baseline recommendation instead of forcing a confident-sounding answer. This is the single most differentiating behavior in the product: a system that can say "I don't know" is more trustworthy than one that always has an opinion, and no naive competing approach will build this in.

A distribution, never a point estimate, for the rival's hidden state

Treating an inferred rival state as verified fact would contradict the project's own core safety principle. Propagating it as a distribution keeps the epistemic honesty consistent everywhere: wider uncertainty about the rival naturally produces wider, more conservative simulated outcomes in Stage 2, and Stage 3 is made structurally more cautious — more likely to fall back to the safe baseline — specifically when that uncertainty is high, rather than trusting a shaky estimate.

Cited real 2026 regulations instead of invented constants

The whole pipeline's credibility rests on the regulatory gate being verifiable ground truth. If a single constant is wrong and a knowledgeable reviewer catches it, every downstream claim becomes suspect by association — so every regulation-mapped constant is commented with the specific article it encodes, and anything not yet independently confirmed is labeled as pending rather than quietly guessed.

4. Current Build Status

In the interest of the whole team working from the same accurate picture, not an aspirational one:

Component	Status
Architecture, data contracts & regulatory constants	Fully specified and documented (context.md)
Stage 1 — Regulatory Gate	Designed, not yet implemented
Rival Energy State Estimator	Designed, not yet implemented
Stage 2 — Monte Carlo Planner	Designed, not yet implemented
Stage 3 — Confidence Gate	Designed, not yet implemented
Stage 4 — LLM Narrator	Not started (by design — waits on Stage 3)
Dashboard UI	Built (React + Three.js), running against representative mock data shaped exactly like the real stage outputs
Backend-to-frontend bridge	Not built yet — planned as a small FastAPI service reusing the same typed models throughout
Real telemetry source	Undecided — leading option is replaying real 2026 FastF1 timing data rather than synthetic data, for credibility

Next up: implementing Stages 1-3 and the estimator as real, tested Python code, then wiring the dashboard to it — at which point the mock data becomes a fallback demo mode rather than the only mode.